



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Jiménez Elizalde Josué

N° de Cuenta: 320334489

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

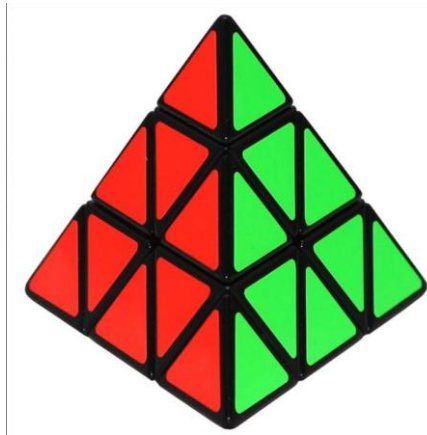
FECHA DE ENTREGA LÍMITE: 11/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

En esta práctica se pidió la elaboración de un rubik (pyraminx) con primitivos que fueron las pirámides triangulares que se elaboraron.



De primer paso se elaboró un triángulo negro para representar el fondo. En código simplemente solo se modificó el color de la primitiva que se tenía antes.

```
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
model = glm::scale(model, glm::vec3(2.5f, 2.5f, 2.5f));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color
del objetos
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh();
```

Como se muestra solo se cambió el color al código inicial, por lo que no se tuvo complejidad.

La segunda parte del problema fue hacer una primitiva, igual una pirámide triangular pero la cual sus caras estén separadas para poder representar los laterales y no desperdiciar recursos.

```

void CrearPiramideTriangularColor()
{
    GLfloat vertices_piramide[] = {
        -0.5f, -0.5f, 0.35f, 1.0f, 0.0f, 0.0f,
        0.5f, -0.5f, 0.35f, 1.0f, 0.0f, 0.0f,
        0.0f, 0.5f, 0.05f, 1.0f, 0.0f, 0.0f,

        0.55f, -0.5f, 0.25f, 0.0f, 1.0f, 0.0f,
        0.05f, -0.5f, -0.65f, 0.0f, 1.0f, 0.0f,
        0.05f, 0.5f, -0.05f, 0.0f, 1.0f, 0.0f,

        -0.55f, -0.5f, 0.25f, 0.0f, 0.0f, 1.0f,
        -0.05f, -0.5f, -0.65f, 0.0f, 0.0f, 1.0f,
        -0.05f, 0.5f, -0.05f, 0.0f, 0.0f, 1.0f,

        -0.55f, -0.60f, 0.25f, 1.0f, 1.0f, 0.0f,
        0.55f, -0.60f, 0.25f, 1.0f, 1.0f, 0.0f,
        0.0f, -0.60f, -0.65f, 1.0f, 1.0f, 0.0f

    };

    MeshColor* piramideColor = new MeshColor();
    piramideColor->CreateMeshColor(vertices_piramide, 72); //12 vértices * 6
floats
    meshColorList.push_back(piramideColor);
}

```

Para todo este código se tomo en cuenta anteriores practicas y se modificó los vértices para que se vieran separados, igual el color se tuvo diferente.

A la hora de implementarlo se utilizo otros shader para implementar los colores.

```

shaderList[1].useShader();
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();

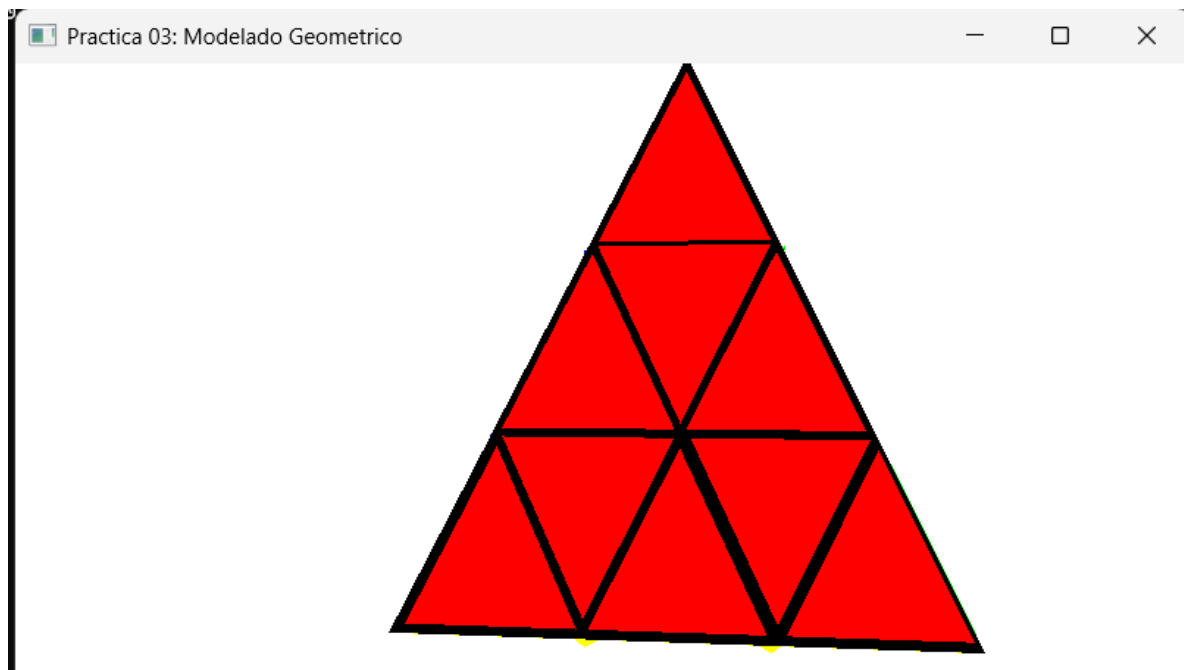
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.8f, -4.0f));
model = glm::scale(model, glm::vec3(0.75f, 0.75f, 0.75f));
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[0]->RenderMeshColor();

```

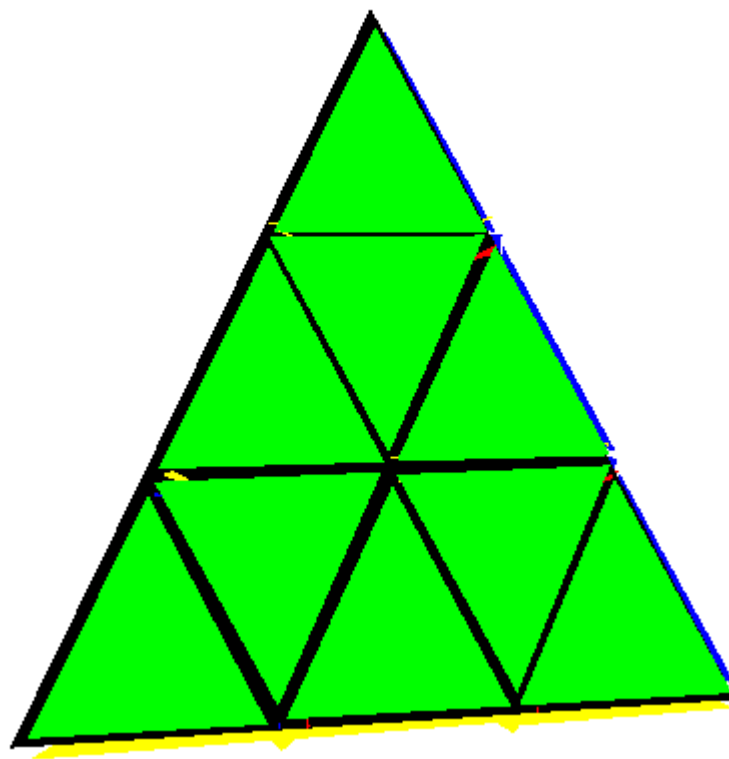
Se muestra en ejemplo de la implementación de este triángulo.

Ejemplo de la implementación

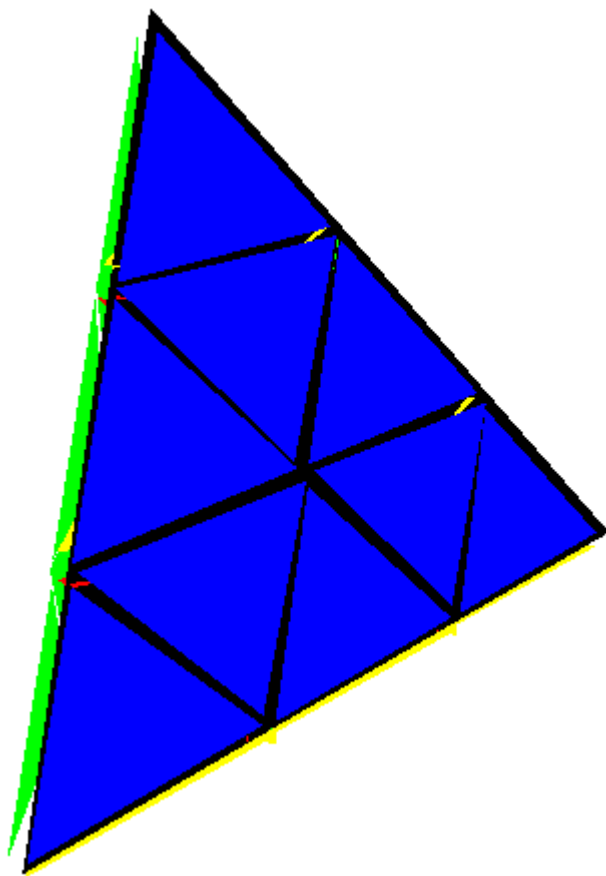
Cara Roja:



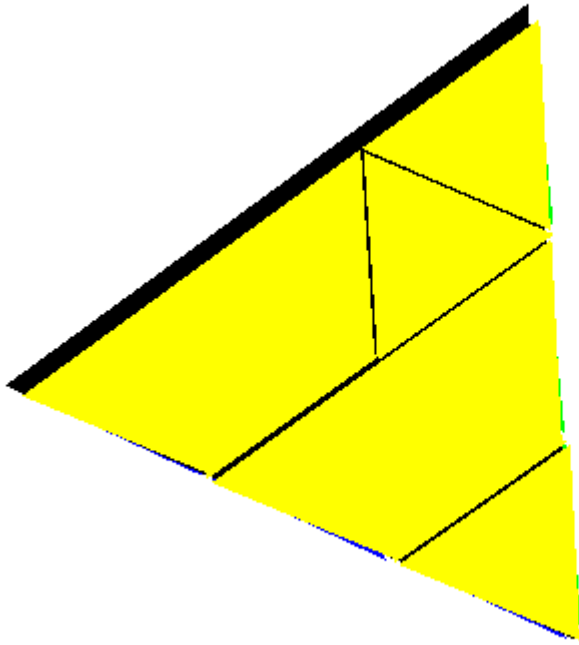
Cara Verde



Cara Azul



Cara Amarilla.



Como se ve en la implementación se llegó a la resolución de problema que se nos propuso.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Se tuvo complicaciones para el ángulo de inclinación para colocarlos en la pirámide negra, se tuvo que ir probando uno por uno hasta llegar a una solución factible o de correcta visualización. Fuera de eso la creación de las primitivas no se tuvo complicación.

3.- Conclusión:

La práctica en cuestión de complejidad aumentó bastante, no para los conocimientos obtenidos ya que se tienen. Pero si se tuvieron más complicaciones que otras prácticas y se complicó el entender el concepto de los ángulos y las rotaciones. Se llegó a la solución.

Bibliografía en formato APA

LearnOpenGL - Shaders. (s. f.). <https://learnopengl.com/Getting-started/Shaders>